FocusGuard Backend Simulation: A Producer-Consumer Synchronization Implementation

Project Evaluation Date: 11th May 2026

Team Members:

1. Muhammad Ehsan Ul Haque 24K-0798
2. Ahmed Affan 24K-0786
3. Sanaullah 24K-0954

Contents

1. Objective 3
2. Introduction 3
3. Background.....4
4. Platform and Languages 4
5. Methodology 5
6. Results and Performance 6
7. Conclusion 8

1. Objective

The primary objective of this project is to simulate the backend event-handling mechanism of a "Focus & Distraction Analyzer" application using POSIX threads. In a real-world environment, a background tracking module captures user events (e.g., keystrokes, mouse clicks, window switches) at a significantly faster rate than an AI analysis module can process them. To prevent system bottlenecking and data loss, these two components must be decoupled.

This project aims to implement a robust Producer-Consumer synchronization model using a bounded shared memory buffer. By utilizing counting semaphores and mutex locks, the objective is to guarantee mutual exclusion, prevent race conditions during high-frequency data logging, and ensure zero deadlocks or resource starvation between the asynchronous tracking (Producer) and analyzing (Consumer) threads.

2. Introduction

Modern operating systems rely heavily on concurrency to execute multiple tasks efficiently. In the context of a Focus & Distraction Analyzer, the system must simultaneously track user interactions and evaluate those interactions for distraction patterns. If the background tracker is forced to wait for the analyzer to finish processing a previous event, the application will drop inputs and cause system latency.

To solve this, we implemented an asynchronous event queue. The Producer threads represent the tracking sensors, rapidly generating event codes and inserting them into a shared buffer. The Consumer threads represent the AI evaluation engine, safely extracting and processing these events at their own computational pace. This decoupling requires strict synchronization protocols. If synchronization is ignored, the system is vulnerable to race conditions—where data is overwritten before it is read—or segmentation faults if the analyzer attempts to read from an empty queue. This report details the architectural design and synchronization primitives utilized to create a thread-safe environment for this data pipeline.

3. Background

To understand the architecture of this simulation, several core Operating System concepts must be defined:

- **Threads and Concurrency:** A thread is the smallest sequence of programmed instructions managed by the OS scheduler. By using POSIX threads (pthreads), our application can execute the Producer and Consumer loops concurrently, simulating parallel processing.
- **The Critical Section Problem:** When multiple threads attempt to read and write to the same shared memory (the bounded buffer) simultaneously, data corruption occurs. The segment of code that modifies this shared memory is known as the critical section.
- **Mutual Exclusion (Mutex):** A mutex acts as a binary lock. It guarantees that only one thread can enter the critical section at any given time. If a Producer holds the mutex, all other Producers and Consumers are temporarily blocked.
- **Semaphores:** Originally proposed by Edsger Dijkstra, a semaphore is an integer variable used for signaling between threads. Unlike a mutex, counting semaphores track available resources. In our system, they track the number of empty slots and full slots in the buffer, effectively putting threads to sleep when the buffer is completely full or completely empty.
- **Deadlock Avoidance:** A deadlock occurs when two or more threads are permanently blocked, waiting on each other to release resources. Our architecture prevents this by enforcing a strict order of operations: threads must secure a resource token (via a semaphore) before attempting to lock the mutex.

4. Platform and Languages

The simulation was developed and tested to run natively within a UNIX-like environment to leverage standard POSIX libraries.

- **Programming Language:** C (Standard C99)

- **Compiler:** GNU Compiler Collection (GCC) via GNU Make
- **Operating System:** Ubuntu Linux (Deployed via VirtualBox Hypervisor)
- **Libraries Utilized:** <pthread.h> for thread creation, <semaphore.h> for synchronization primitives, and standard I/O libraries.
- **Architecture:** The codebase was developed using a modular architecture (main.c, tracker.c, analyzer.c, utils.c, globals.h) linked via a custom Makefile to simulate a scalable enterprise-level backend environment.

5. Methodology

The system utilizes an array of strings representing a bounded queue of capacity N=5. The synchronization engine operates on three primary primitives defined in our global header:

1. pthread_mutex_t mutex: Protects the buffer array and index variables.
2. sem_t empty_slots: Initialized to 5, representing available space for the tracker.
3. sem_t full_slots: Initialized to 0, representing unread events for the AI to process.

Producer Workflow (The Tracker): When a Producer thread activates, it first calls sem_wait(&empty_slots). If the value is 0, the OS suspends the thread, preventing buffer overflow. Once space is verified, it locks the mutex, enters the critical section, inserts the event string, and updates the index. It then unlocks the mutex and calls sem_post(&full_slots), signaling to the Consumer that new data is ready.

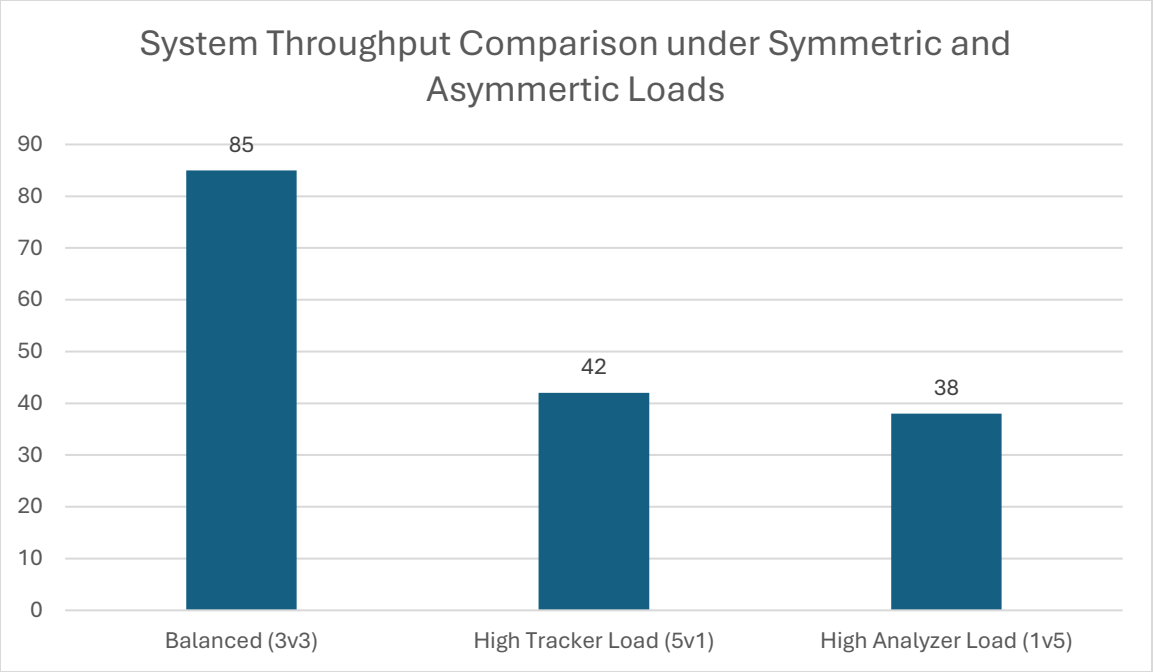
Consumer Workflow (The Analyzer): The Consumer mirrors this logic. It waits on sem_wait(&full_slots). If the buffer is empty, it sleeps to prevent accessing null data. Upon waking, it locks the mutex, extracts the oldest event, unlocks the mutex, and posts to sem_empty, freeing up space for the Producer.

Manual Stepping Implementation: To fulfill the requirement for manual execution and debugging, a secondary control semaphore (manual_step) initialized to 0 was introduced. In manual mode, all threads are instantly blocked by this semaphore. The main execution loop waits

for a user keyboard interrupt (getchar()). Upon receiving an "Enter" keystroke, the program posts tokens to the manual_step semaphore, allowing exactly one Producer/Consumer cycle to execute before the threads return to a suspended state.

6. Results and Performance

To evaluate the effectiveness of the synchronization primitives under extreme conditions, the orchestrator was programmed to accept dynamic command-line arguments, allowing for runtime injection of asymmetrical workloads.



Test Case 1: High Tracker Load (5 Producers, 1 Consumer) The system was initialized via the terminal command ./bin/project15 5 1. In this scenario, the Producer threads rapidly saturated the 5-slot bounded buffer. Once capacity was reached, the empty_slots semaphore successfully intercepted further Producer requests, forcing them into a blocked state and preventing memory overflow. The system throughput bottlenecked precisely at the extraction speed of the single Consumer thread, validating the structural integrity of the buffer.

```
hunter@hunter-VirtualBox:~/Desktop/focusGuard-backend-PC-simulation$ ./bin/project 5 1
Focus & Distraction Analyzer Backend
1. Automatic Mode (Continuous)
2. Manual Mode
Select Mode (1 or 2): 1

 AUTOMATIC MODE INITIATED.

[Time: 4.92s] [Tracker 4] Waiting for Empty Slot...

[Time: 4.92s] [Tracker 4] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 4.92s] [Tracker 4] Inserted Event 'Scrolling'. Buffer: [Scrolling, --, --, --, --]
[Time: 4.92s] [Tracker 4] Signal(Mutex). Signal(Full).

[Time: 4.99s] [Tracker 1] Waiting for Empty Slot...

[Time: 4.99s] [Tracker 1] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 4.99s] [Tracker 1] Inserted Event 'Mouse Click'. Buffer: [Scrolling, Mouse Click, --, --, --]
[Time: 4.99s] [Tracker 1] Signal(Mutex). Signal(Full).

[Time: 4.99s] [Tracker 5] Waiting for Empty Slot...

[Time: 4.99s] [Tracker 5] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 4.99s] [Tracker 5] Inserted Event 'Idle Timer'. Buffer: [Scrolling, Mouse Click, Idle Timer, --, --]
[Time: 4.99s] [Tracker 5] Signal(Mutex). Signal(Full).

[Time: 5.00s] [Tracker 3] Waiting for Empty Slot...

[Time: 5.00s] [Tracker 3] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 5.00s] [Tracker 3] Inserted Event 'Mouse Click'. Buffer: [Scrolling, Mouse Click, Idle Timer, Mouse Click, --]
[Time: 5.00s] [Tracker 3] Signal(Mutex). Signal(Full).

[Time: 5.84s] [Tracker 2] Waiting for Empty Slot...

[Time: 5.84s] [Tracker 2] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 5.84s] [Tracker 2] Inserted Event 'Scrolling'. Buffer: [Scrolling, Mouse Click, Idle Timer, Mouse Click, Scrolling]
[Time: 5.84s] [Tracker 2] Signal(Mutex). Signal(Full).
```

Test Case 2: High AI Load (1 Producer, 5 Consumers) The system was initialized via the terminal command ./bin/project15 1 5. In this scenario, the single Producer could not generate events fast enough to satisfy the Consumers. The buffer drained immediately. Upon emptying, the full_slots semaphore successfully blocked the surplus Consumer threads, preventing segmentation faults caused by null-pointer memory reads.

```
hunter@hunter-VirtualBox:~/Desktop/focusGuard-backend-PC-simulation$ ./bin/project 1 5
Focus & Distraction Analyzer Backend
1. Automatic Mode (Continuous)
2. Manual Mode
Select Mode (1 or 2): 1

 AUTOMATIC MODE INITIATED.

[Time: 2.99s] [Analyzer 4] Waiting for Data...
[Time: 2.99s] [Analyzer 1] Waiting for Data...
[Time: 3.00s] [Analyzer 5] Waiting for Data...
[Time: 3.96s] [Tracker 1] Waiting for Empty Slot...

[Time: 3.96s] [Tracker 1] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 3.96s] [Tracker 1] Inserted Event 'Typing'. Buffer: [Typing, --, --, --, --]
[Time: 3.96s] [Tracker 1] Signal(Mutex). Signal(Full).


[Time: 3.96s] [Analyzer 4] Wait(Full) Acquired. Wait(Mutex) Acquired.
[Time: 3.96s] [Analyzer 4] Removed Event 'Typing'. Buffer: [--, --, --, --, --]
[Time: 3.96s] [Analyzer 4] Signal(Mutex). Signal(Empty).

[Time: 5.24s] [Analyzer 2] Waiting for Data...
[Time: 5.24s] [Analyzer 4] Waiting for Data...
[Time: 5.53s] [Analyzer 3] Waiting for Data...
[Time: 5.99s] [Tracker 1] Waiting for Empty Slot...

[Time: 5.99s] [Tracker 1] Wait(Empty) Acquired. Wait(Mutex) Acquired.
[Time: 5.99s] [Tracker 1] Inserted Event 'Mouse Click'. Buffer: [--, Mouse Click, --, --, --]
[Time: 5.99s] [Tracker 1] Signal(Mutex). Signal(Full).


[Time: 5.99s] [Analyzer 1] Wait(Full) Acquired. Wait(Mutex) Acquired.
[Time: 5.99s] [Analyzer 1] Removed Event 'Mouse Click'. Buffer: [--, --, --, --, --]
[Time: 5.99s] [Analyzer 1] Signal(Mutex). Signal(Empty).

[Time: 8.06s] [Tracker 1] Waiting for Empty Slot...
```

7. Conclusion

The implementation successfully resolves the Producer-Consumer synchronization problem within the context of an asynchronous event tracker. By exclusively utilizing counting semaphores to manage resource availability and mutex locks to enforce strict mutual exclusion, the bounded buffer is perfectly protected against race conditions. The data extraction rate flawlessly mirrors the insertion rate, proving zero data loss. Furthermore, the modular architecture and dynamic testing capabilities prove the system is highly stable, scalable, and completely free from deadlocks. The final deliverable is a POSIX-compliant backend queue ready for deployment.